# THƯ VIỆN



1

# TỔNG QUAN

| | |
|---|---|
| System | Object, Type, Int32, Boolean, Array, String, Random, Math, ... |
| Collections | ArrayList, BitArray, Hashtable, SortedList, Stack, Queue, ... |
| Drawing | Bitmap, Color, Font, Pen, Point, Rectangle, Region, ... |
| Design | |
| Drawing2D | |
| Imaging | |
| Printing | |
| Text | |
| IO | Stream, File, Directory, FileStream, StreamReader, StreamWriter, ... |
| Net | WebClient, WebRequest, WebResponse, ... |
| Reflection | Assembly, FieldInfo, MethodInfo, PropertyInfo, EventInfo, ... |
| Runtime | |
| InteropServices | DllImportAttribute, StructLayoutAttribute, ... |
| Remoting | |
| Serialization | Formatter, ISerializable, ... |
| Security | CodeAccessPermission, PermissionSet, SecurityException, ... |
| Text | StringBuilder, Encoder, Decoder, ... |
| Threading | Thread, Monitor, Mutex, Timer, ... |
| Web | HttpRequest, HttpRespose, .. |
| Windows | |
| Forms | Form, Button, CheckBox, Menu, RadioButton, ListBox, ... |
| XML | XmlDoxument, XmlElement, XmlReader, ... |



2

# Math

```
public sealed class Math {
        public const double PI = 3.14....;
        public const double E = 2.71...;

        public static T Abs(T val);                  // T là kiểu: sbyte, short, int, long, float,
        double, decimal
        public static T Sign(T val);
        public static T1 Min(T1 x, T1 y);     // T1, T là kiểu: byte, ushort, uint, ulong
        public static T1 Max(T1 x, T1 y);

        public static double Round(double x);
        public static double Floor(double x);
        public static double Ceiling(double x);

        public static double Sqrt(double x);
        public static double Pow(double x, double y);          // x^y
        public static double Exp(double x)                     // e^x
        public static double Log(double x);                    // log_e x
        public static double Log10(double x);                                  // log_10 x
        public static double Sin(double x);                    // góc tính theo radian
        public static double Cos(double x);
        public static double Tan(double x);
        public static double Asin(double x);
        public static double Acos(double x);
        public static double Atan(double x);
}
```

3

3

# Random

```
public class Random {
        public Random();
        public Random(int seed);
        public virtual int Next();          // 0 <= res < int.MaxVal
        public virtual int Next(int x);    // 0 <= res < x
        public virtual int Next(int x, int y);        // x <= res < y
        public virtual double NextDouble();       // 0.0 <= res < 1.0
        public virtual void NextBytes(byte[] b); // các phần tử của mảng b là
        số ngẫu nhiên
}
```

4

4

# String

```
public sealed class String : ICloneable, IComparable, IEnumerable {
        public String(char[] a);
        public String(char[] a, int start, int len);

        public static string Copy(string s);
        public static string Format(String s, params object[]);
        public static string Join(string separator, string[] parts);
        public static bool operator ==(bool a, bool b);
        public static bool operator !=(bool a, bool b);

        public int Length { get; }
        public char this[int i] { get; }

        public override bool Equals(object o);
        public int CompareTo(object o);
        public static int CompareOrdinal(string a, string b);
        public void CopyTo(int from, char[] dest, int to, int len);
        public bool StartsWith(string s);
        public bool EndsWith(string s);
        public int IndexOf(T s);   // T ... string, char
        public int LastIndexOf(T s);
        public string Substring(int pos);
        public string Substring(int pos, int len);
        public string[] Split(params char[]);
        public char[] ToCharArray();
        ...
}
```



5

# Chuyển đổi giữa string và số

```
public sealed class Convert {          // namespace System
        public static string ToString(T x);          // T là kiểu số bất kỳ
        public static bool       ToBoolean(string s);
        public static byte       ToByte(string s);
        public static sbyte      ToSByte(string s);
        public static char       ToChar(string s);
        public static short      ToInt16(string s);
        public static int        ToInt32(string s);
        public static long       ToInt64(string s);
        public static ushort ToUInt16(string s);
        public static uint       ToUInt32(string s);
        public static ulong ToUInt64(string s);
        public static float      ToSingle(string s);
        public static double ToDouble(string s);
        public static decimal ToDecimal(string s);
        ...
}

        string s = Convert.ToString(123);        // "123"
        s = 123.ToString();   // "123"

        int i = Convert.ToInt32(s);      // 123
```



6

# StringBuilder

```
public sealed class StringBuilder {        // namespace System.Text
    public StringBuilder();
    public StringBuilder(int initCapacity);
    public StringBuilder(string s);
    public StringBuilder(string s, int from, int len);

    public int Length { get; set; }
    public int Capacity { get; set; }
    public char this[int i] { get; set; }

    public StringBuilder Append(T x);   // T là kiểu string hoặc kiểu số
    public StringBuilder Insert(int pos, T x);
    public StringBuilder Remove(int pos, int len);
    public StringBuilder Replace(T x, T y);        // T  là kiểu string, char
    public bool Equals(object x);
    public string ToString();
    ...
}
```

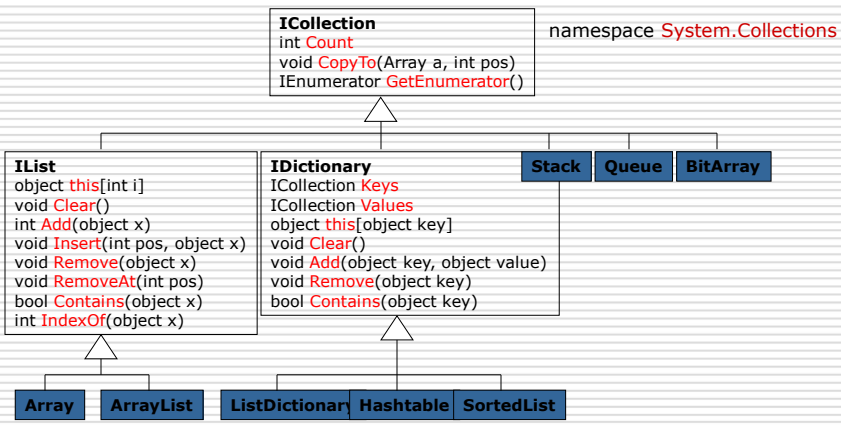
**Ví dụ**

```
StringBuilder b = new StringBuilder("A.C");
b.Insert(2, "B.");
b.Replace('.', '/');
Console.WriteLine(b.ToString());    // A/B/C
```

7

7

# Collection class




8

8

# Array

Tất cả các mảng đều kế thừa từ lớp Array

```
public abstract class Array : IList, ICloneable, IEnumerable{  // namespace System
    public int Length { get; }// tổng số các phần tử trong các chiều
    public int Rank { get; }  // tổng số các chiều

    public static void Clear(Array a, int pos, int length);
    public static void Copy(Array src, int from, Array dst, int to, int len);
    public static int IndexOf(Array a, object x);
    public static int LastIndexOf(Array a, object x);
    public static void Reverse(Array a);
    public static void Sort(Array a);
    public static int BinarySearch(Array a, object x);

    public virtual object Clone();
    public virtual bool Equals(object x);
    public int GetLength(int dimension);
    ...
}
```

**Ví dụ**

```
int[] a = {17, 3, 5, 9, 6};
Array.Sort(a);
Array.Reverse(a);
foreach (int x in a) Console.Write("{0} ", x); // 17 9 6 5 3
```

9

9

# ArrayList

Mảng động

```
public class ArrayList : IList, IEnumerable, ICloneable {       // namespace
    System.Collections
    public ArrayList();
    public ArrayList(int initCapacity);
    public ArrayList(ICollection c);

    public virtual int          Count { get; }
    public virtual int          Capacity { get; set; }
    public virtual object       this[int i] { get; set; }

    public virtual void         Clear();
    public virtual int          Add(object x);
    public virtual void         Insert(int pos, object x);
    public virtual void         Remove(object x);
    public virtual void         RemoveAt(int pos);
    public virtual object       Clone();
    public virtual bool         Equals(object x);
    public virtual bool         Contains(object x);
    public virtual int          IndexOf(object x);
    public virtual void         Reverse();
    public virtual void         Sort();
    public virtual int          BinarySearch(object x);
    ...
}
```

10

10

# Ví dụ ArrayList

```
using System;
using System.Collections;

class Test {

      static void Main(string[] arg) {
            ArrayList a = new ArrayList();
            foreach (string s in arg) a.Add(s);
            a.Sort();
            Console.Write("a = ");
            for (int i = 0; i < a.Count; i++) Console.Write("{0} ", a[i]);
            Console.WriteLine();
            int pos = a.BinarySearch("Tom");
            if (pos >= 0)
                  Console.WriteLine("Tom found at {0}", pos);
            else
                  Console.WriteLine("Tom not found");
      }
}
```

Aufruf: Test John Tom Alice Charly

Ausgabe:

```
a = Alice Charly John Tom
Tom found at 3
```

11

11

# ListDictionary

**Tương tự như danh sách liên lết**

```
public class ListDictionary : IDictionary, IEnumerable {  //
      System.Collections.Specialized
      public ListDictionary();
      public int Count { get; }
      public ICollection Keys { get; }
      public ICollection Values { get; }
      public object this[object key] { get; set; }

      public void Clear();
      public void Add(object key, object value);
      public void Remove(object key);
      public bool Contains(object key);
      public virtual bool Equals(object x);
      public IDictionaryEnumerator GetEnumerator();
      ...
}
```

**Ví dụ**

```
      ListDictionary population = new ListDictionary();
      population["Vienna"] = 1592600;
      population["London"] = 7172000 ;
      population["Paris"] = 2305000;
      ...
      foreach (DictionaryEntry x in tab) Console.WriteLine("{0} = {1}", x.Key,
      x.Value);
```

12

12

# Hashtable

```
public class Hashtable : IDictionary, IEnumerable, ISerializable, ICloneable { //
        System.Collections
        public HashTable();
        public virtual object Clone();
        public virtual bool ContainsKey(object key);
        public virtual bool ContainsValue(object value);
}
```

**Ví dụ**

```
        Hashable population = new Hashtable();
        population["Vienna"] = 1592600;
        population["London"] = 7172000;
        population["Paris"] = 2305000;
        ...
        foreach (DictionaryEntry x in population) Console.WriteLine("{0} = {1}", x.Key,
        x.Value);
```

13

13

# SortedList

**Là mảng và được sắp xếp bởi Key**

```
public class SortedList : IDictionary, IEnumerable, ICloneable {     //
        System.Collections
        public SortedList();        // variants exist
        //... gleiche Members wie Hashtable; zusätzlich:
        public virtual int Capacity { get; set; }
        public virtual object GetKey(int i);  // returns Key[i]
        public virtual object GetByIndex(int i);          // returns Value[i]
        public virtual void SetByIndex(int i, object value);
        public virtual IList GetKeyList();
        public virtual IList GetValueList();
        public virtual int IndexOfKey(object key);
        public virtual int IndexOfValue(object value); // returns index of the first
        occurrence of value
        public virtual void RemoveAt(int i);
}
```

**Ví dụ**

```
        SortedList population = new SortedList();
        population["Vienna"] = 1592600;
        population["London"] = 7172000;
        ...
        foreach (DictionaryEntry x in population) Console.WriteLine("{0} = {1}", x.Key,
        x.Value);
        // in ra các phần tử được sắp xếp theo Key
```

14

14

# Stack

```
public class Stack : ICollection, IEnumerable, ICloneable { // System.Collections
        public Stack();
        public Stack(int initCapacity);
        public virtual int Count { get; }
        public virtual void Clear();
        public virtual void Push(object x);
        public virtual object Pop();
        public virtual object Peek();
        public virtual bool Contains(object x);
        public virtual object Clone();
        public virtual IEnumerator GetEnumerator();
        public virtual object[] ToArray();
        ...
}
```

**Ví dụ**

```
Stack s = new Stack();
for (int i = 0; i <= 20; i++) s.Push(i);
while (s.Count > 0) Console.Write("{0} ", (int)s.Pop());      // trả về 20 19 18 17
... 0
```

15

15

# Queue

```
public class Queue : ICollection, IEnumerable, ICloneable { // System.Collections
        public Queue();
        public virtual int Count { get; }
        public virtual void Clear();
        public virtual void Enqueue(object x);
        public virtual object Dequeue();
        public virtual object Peek();
        public virtual bool Contains(object x);
        public virtual object Clone();
        public virtual IEnumerator GetEnumerator();
        public virtual object[] ToArray();
        ...
}
```

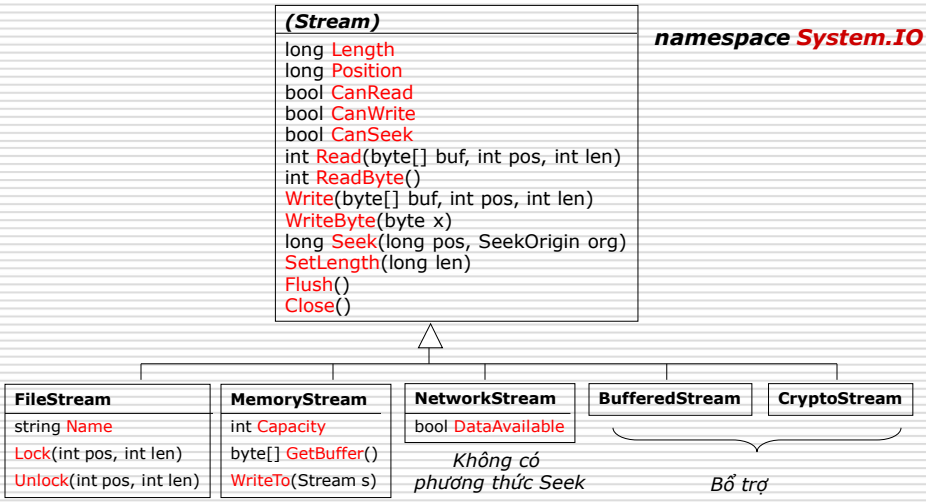
**Ví dụ**

```
Queue q = new Queue();
q.Enqueue("one");
q.Enqueue("two");
q.Enqueue("three");
foreach (string s in q) Console.Write(s + " ");  // one two three
// q vẫn đầy
Console.WriteLine();
while (q.Count > 0) Console.Write((string)q.Dequeue() + " ");   // one two three
// q rỗng
```

16

16

# Stream class






17

# Ví dụ FileStream

```
using System;
using System.IO;

class Test {

    static void Copy(string from, string to, int pos) {
        try {
            FileStream sin = new FileStream(from, FileMode.Open);
            FileStream sout = new FileStream(to, FileMode.Create);
            sin.Seek(pos, SeekOrigin.Begin);        // di chuyển tới cuối file
            int ch = sin.ReadByte();
            while (ch >= 0) {
                sout.WriteByte((byte)ch);
                ch = sin.ReadByte();
            }
            sin.Close();
            sout.Close();
        } catch (FileNotFoundException e) {
            Console.WriteLine("-- file {0} not found", e.FileName);
        }
    }

    static void Main(string[] arg) {
        Copy(arg[0], arg[1], 10);
    }
}
```



18